cmd\internal\kademlia\table.go

```go
package kademlia

import (
    "bytes"
    "fmt"
    "math/bits"
    "net"
    "net/netip"
    "sort"
    "sync"

    "github.com/ethereum/go-ethereum/p2p/discover/v4wire"
    "github.com/ethereum/go-ethereum/p2p/enode"
)

// ID는 go-ethereum의 노드 식별자를 그대로 사용해 Kademlia 거리 계산을 동일한 타입으로 다루게 한다.
type ID = enode.ID

// Node는 라우팅 테이블이 추적하는 참가자 한 명을 나타내며, Kademlia 버킷이 저장하는 식별자와 연락처 정보를 그대로 담는다.
type Node struct {
    ID     ID
    Pubkey v4wire.Pubkey
    IP     netip.Addr
    UDP    uint16
    TCP    uint16
    Label  string
}

// AddrPort는 각 노드가 UDP로 연락 가능한 엔드포인트를 돌려주어, Kademlia 버킷 항목이 주소/포트를 저장하는 방식을 재현한다.
func (n Node) AddrPort() netip.AddrPort {
    if !n.IP.IsValid() {
        return netip.AddrPort{}
    }
    return netip.AddrPortFrom(n.IP, n.UDP)
}

// String은 디버깅 시 어느 논리적 거리에 어떤 노드가 있는지 읽기 쉽게 문자열로 표현한다.
func (n Node) String() string {
    if n.Label != "" {
        return fmt.Sprintf("%s(%s:%d)", n.Label, addrString(n.IP), n.UDP)
    }
    return fmt.Sprintf("%s:%d", addrString(n.IP), n.UDP)
}

// addrString은 버킷에 저장된 주소를 로깅할 때 보기 좋게 만들어 주는 보조 함수다.
func addrString(ip netip.Addr) string {
    if !ip.IsValid() {
        return "<nil>"
    }
```

```go
    return ip.String()
}

// Table은 Kademlia 테이블의 핵심 상태(로컬 ID와 XOR 거리 접두어로 나뉜 버킷 배열)를 가지고 있으며, 동시에 갱신되므로 뮤텍스로 보호한다.
type Table struct {
    mu         sync.RWMutex
    local      ID
    bucketSize int
    buckets    [idBits]*bucket
}

// idBits와 idBytes는 식별자 공간의 비트 수(이더리움은 256비트)를 정의하고, defaultK는 표준 K 값(버킷 용량)을 따른다.
const (
    idBits   = 256
    idBytes  = idBits / 8
    defaultK = 16
)

// bucket은 특정 거리 접두어에 속한 노드 리스트를 시간 순서로 유지하며, 앞쪽이 가장 오래된 항목이어서 용량을 넘기면 쉽게 버릴 수 있다.
type bucket struct {
    nodes []*Node
}

// NewTable은 지정된 로컬 ID를 중심으로 비어 있는 테이블을 만들고, 제공된 버킷 크기를 K 값으로 사용한다(0이면 기본값 사용).
func NewTable(local ID, bucketSize int) *Table {
    if bucketSize <= 0 {
        bucketSize = defaultK
    }
    return &Table{
        local:      local,
        bucketSize: bucketSize,
    }
}

// ID는 호출자가 XOR 거리를 계산할 때 로컬 식별자를 재사용할 수 있도록 반환한다.
func (rt *Table) ID() ID {
    return rt.local
}

// AddNode는 Kademlia 버킷에 노드를 추가하거나 이미 있으면 최신으로 갱신한다.
func (rt *Table) AddNode(n Node) (added bool) {
    if (n.ID == ID{}) && (n.Pubkey != (v4wire.Pubkey{})) {
        n.ID = n.Pubkey.ID() // 공개키만 있는 경우 Kademlia ID를 재계산한다.
    }
    if n.ID == rt.local || n.ID == (ID{}) {
        return false // 자기 자신이나 미지의 노드는 버킷에 넣지 않는다.
    }
    idx := bucketIndex(rt.local, n.ID)
    if idx < 0 {
        return false // XOR 거리가 0이면 버킷이 존재하지 않는다.
```

```go
    }
    rt.mu.Lock()
    defer rt.mu.Unlock()
    b := rt.getBucket(idx)
    for i, existing := range b.nodes {
        if existing.ID == n.ID {
            b.moveToTail(i)      // 기존 엔트리는 최신으로 갱신해 LRU 순서를 맞춘다.
            existing.IP = n.IP   // 주소 정보를 새 값으로 덮어쓴다.
            existing.UDP = n.UDP // UDP 포트 정보도 최신화한다.
            existing.TCP = n.TCP // TCP 포트 역시 동기화한다.
            if n.Pubkey != (v4wire.Pubkey{}) {
                existing.Pubkey = n.Pubkey // 공개키가 있으면 최신 값으로 교체한다.
            }
            if n.Label != "" {
                existing.Label = n.Label // 디버깅 라벨도 유지 보수한다.
            }
            return false
        }
    }
    nodeCopy := n
    if len(b.nodes) >= rt.bucketSize {
        copy(b.nodes[0:], b.nodes[1:])      // 버킷이 가득 찼으면 가장 오래된 항목을 밀어낸다.
        b.nodes[len(b.nodes)-1] = &nodeCopy // 새 노드를 맨 뒤(가장 최신)에 배치한다.
        return true
    }
    b.nodes = append(b.nodes, &nodeCopy) // 여유가 있으면 그냥 맨 뒤에 추가한다.
    return true
}

// RemoveNode는 테이블에서 특정 노드를 제거한다.
func (rt *Table) RemoveNode(id ID) bool {
    if id == rt.local {
        return false // 라우팅 테이블은 자기 자신을 지우지 않는다.
    }
    idx := bucketIndex(rt.local, id)
    if idx < 0 {
        return false // 버킷이 없으면 자동으로 실패한다.
    }
    rt.mu.Lock()
    defer rt.mu.Unlock()
    b := rt.getBucket(idx)
    for i, existing := range b.nodes {
        if existing.ID == id {
            b.nodes = append(b.nodes[:i], b.nodes[i+1:]...) // 해당 항목을 슬라이스에서 제거한다.
            return true
        }
    }
    return false
}

// FindnodeByID는 대상 ID에 가장 가까운 노드들을 XOR 거리 기준으로 반환한다.
func (rt *Table) FindnodeByID(target ID, max int) []Node {
```

```go
    if max <= 0 {
        max = rt.bucketSize // 요청이 0이면 기본 K 값을 사용한다.
    }
    rt.mu.RLock()
    defer rt.mu.RUnlock()
    var collected []*Node
    for i := range rt.buckets {
        if rt.buckets[i] == nil {
            continue // 비어 있는 버킷은 건너뛴다.
        }
        collected = append(collected, rt.buckets[i].nodes...) // 모든 버킷을 한 리스트로 합친다.
    }
    if len(collected) == 0 {
        return nil // 테이블이 비어 있으면 빈 결과를 돌려준다.
    }
    sort.Slice(collected, func(i, j int) bool {
        return compareDistance(collected[i].ID, collected[j].ID, target) < 0 // XOR 거리가 가까운 순으로 정렬한다.
    })
    if len(collected) > max {
        collected = collected[:max] // 필요한 개수만큼 자른다.
    }
    out := make([]Node, len(collected))
    for i, n := range collected {
        out[i] = *n // 포인터 목록을 값 목록으로 변환한다.
    }
    return out
}

// Len은 테이블에 저장된 총 노드 수를 돌려준다.
func (rt *Table) Len() int {
    rt.mu.RLock()
    defer rt.mu.RUnlock()
    total := 0
    for _, b := range rt.buckets {
        if b != nil {
            total += len(b.nodes) // 각 버킷의 노드 수를 모두 더해 전체 테이블 크기를 구한다.
        }
    }
    return total
}

func (rt *Table) Buckets() []BucketInfo {
    rt.mu.RLock()
    defer rt.mu.RUnlock()
    infos := make([]BucketInfo, 0, len(rt.buckets))
    for i, b := range rt.buckets {
        if b == nil || len(b.nodes) == 0 {
            continue // 비어 있는 버킷은 통계에서 생략한다.
        }
        infos = append(infos, BucketInfo{Index: i, Count: len(b.nodes)})
    }
```

```go
    sort.Slice(infos, func(i, j int) bool { return infos[i].Index < infos[j].Index }) // 인덱스 순으로 정렬해 시각화한다.
    return infos
}

// BucketInfo는 디버그용으로 버킷 인덱스와 노드 수를 묶어서 보여준다.
type BucketInfo struct {
    Index int
    Count int
}

func (rt *Table) ForEach(fn func(Node) bool) {
    rt.mu.RLock()
    defer rt.mu.RUnlock()
    for _, b := range rt.buckets {
        if b == nil {
            continue // 비어 있는 버킷은 건너뛴다.
        }
        for _, n := range b.nodes {
            if !fn(*n) {
                return // 콜백이 false를 반환하면 바로 중단해 탐색 비용을 줄인다.
            }
        }
    }
}

func (rt *Table) Snapshot() []Node {
    rt.mu.RLock()
    defer rt.mu.RUnlock()
    var nodes []Node
    for _, b := range rt.buckets {
        if b == nil {
            continue
        }
        for _, n := range b.nodes {
            nodes = append(nodes, *n) // 버킷 전체를 값 형태로 복사해 외부에서 안전하게 읽게 한다.
        }
    }
    return nodes
}

func (rt *Table) getBucket(idx int) *bucket {
    if rt.buckets[idx] == nil {
        rt.buckets[idx] = &bucket{} // 필요한 순간에 버킷을 생성하는 지연 할당을 사용한다.
    }
    return rt.buckets[idx]
}

func (b *bucket) moveToTail(i int) {
    if i < 0 || i >= len(b.nodes)-1 {
        return // 이미 마지막 노드면 이동이 필요 없다.
    }
    node := b.nodes[i]
```

```go
    copy(b.nodes[i:], b.nodes[i+1:]) // 지정 위치를 삭제하고 뒤 요소들을 앞으로 당긴다.
    b.nodes[len(b.nodes)-1] = node   // 꺼낸 노드를 맨 뒤에 붙여 최신으로 만든다.
}

func bucketIndex(local, remote ID) int {
    xor := xorID(local, remote) // 두 ID의 XOR 결과를 이용해 어느 버킷에 속할지 결정한다.
    for i := 0; i < len(xor); i++ {
        if xor[i] != 0 {
            return (len(xor)-i-1)*8 + (bits.Len8(xor[i]) - 1) // 첫 번째 1비트의 위치가 곧 버킷 인덱스다.
        }
    }
    return -1 // 완전히 같은 ID는 버킷이 없으므로 -1을 돌려준다.
}

func compareDistance(a, b, target ID) int {
    da := xorID(a, target) // 대상까지의 XOR 거리를 구한다.
    db := xorID(b, target)
    return bytes.Compare(da[:], db[:]) // 바이트 단위 비교로 더 가까운 순서를 판별한다.
}

// distance는 XOR 결과를 그대로 담아 비교에 사용할 수 있게 만든다.
type distance [idBytes]byte

func xorID(a, b ID) (out distance) {
    ab := a.Bytes()
    bb := b.Bytes()
    for i := 0; i < idBytes; i++ {
        out[i] = ab[i] ^ bb[i] // 각 바이트를 XOR해 Kademlia 공간의 거리 벡터를 만든다.
    }
    return out
}

func (d distance) Bytes() []byte {
    return d[:] // 거리 구조체를 바이트 슬라이스로 노출한다.
}

func FromV4Node(n v4wire.Node) Node {
    var ip netip.Addr
    if len(n.IP) != 0 {
        addr, ok := netip.AddrFromSlice(n.IP)
        if ok {
            ip = addr // v4wire 포맷의 주소를 netip으로 변환한다.
        }
    }
    return Node{
        ID:     n.ID.ID(), // 공개키에서 파생된 ID를 사용한다.
        Pubkey: n.ID,
        IP:     ip,
        UDP:    n.UDP,
        TCP:    n.TCP,
    }
}

```

```go
func ToV4Node(n Node) v4wire.Node {
    var ip net.IP
    if n.IP.IsValid() {
        if n.IP.Is4() {
            v4 := n.IP.As4()
            ip = net.IPv4(v4[0], v4[1], v4[2], v4[3]) // IPv4 주소는 4바이트로 변환한다.
        } else {
            v6 := n.IP.As16()
            ip = net.IP(v6[:]) // IPv6 주소는 16바이트 전체를 사용한다.
        }
    }
    return v4wire.Node{
        IP:  ip,
        UDP: n.UDP,
        TCP: n.TCP,
        ID:  n.Pubkey,
    }
}

```